

Unidad Didáctica

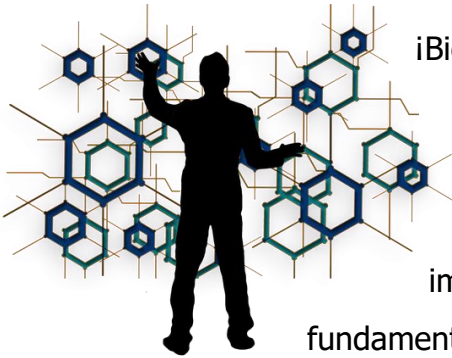
**Modelo Relacional,
Unidad III**

Índice de Contenido

Contenido

Introducción.....	3
Desarrollo de Contenidos.....	4
Etapas del diseño de datos.....	4
Diseño lógico estándar	5
Diseño lógico específico	7
Transformación del Modelo Entidad Relación a Modelo Relacional	8
Entidades Fuertes	9
Entidades Débiles.....	10
Atributos Multivalorados	11
Relación 1: 1	12
Relación 1: N.....	13
Relación M: N	14
Relación n-arios (grado mayor que 2)	15
Teoría de la Normalización	16
Dependencias Funcionales	17
Primera Forma Normal	18
Segunda Forma Normal	19
Tercera Forma Normal.....	20
Forma Normal de Boyce-Codd	21
Cuarta Forma Normal	22
Quinta Forma Normal	23
Bibliografía y Webgrafía	24
Glosario	24

Introducción



¡Bienvenido! En el mundo de la tecnología, el diseño de datos es una de las etapas cruciales en el desarrollo de cualquier proyecto. Desde el momento en que se concibe la idea hasta el momento en que se implementa, el diseño de datos juega un papel fundamental para que todo funcione correctamente.

La primera etapa del diseño de datos es el diseño lógico estándar, donde se establecen las reglas básicas para organizar la información de manera coherente. En esta etapa, se establecen las tablas, los campos y las relaciones entre ellos. La segunda etapa es el diseño lógico específico, donde se ajusta el diseño a las necesidades específicas del proyecto. Uno de los procesos más importantes del diseño de datos es la transformación del Modelo Entidad-Relación a Modelo Relacional. En esta etapa, se convierten las relaciones lógicas en tablas físicas que pueden ser almacenadas en una base de datos. En el modelo relacional, se utilizan conceptos como entidades fuertes y débiles, atributos multivalorados y diferentes tipos de relaciones, como la relación 1:1, 1:N, M:N y n-arias (grado mayor que 2).

La teoría de la normalización es otro aspecto importante del diseño de datos. Se trata de un proceso que busca eliminar la redundancia y asegurar la integridad de los datos en una base de datos. En la normalización se trabajan las dependencias funcionales y se establecen diferentes formas normales, como la primera, segunda, tercera, cuarta y quinta forma normal, y la forma normal de Boyce-Codd.

En resumen, el diseño de datos es una disciplina fundamental en el desarrollo de cualquier proyecto tecnológico que involucre una base de datos. Desde el diseño lógico hasta la normalización, cada etapa del proceso es crucial para asegurar el correcto funcionamiento del sistema y garantizar la integridad de los datos.

Etapas del diseño de datos

El diseño de datos es un proceso esencial en la gestión de la información, ya que permite definir la estructura de la base de datos y la forma en que los datos serán almacenados, organizados y manipulados. Para lograr un diseño de datos eficiente y efectivo, es importante seguir una serie de etapas clave.

La primera etapa es la definición de los requisitos del sistema, en la cual se identifican los datos que se necesitan recopilar y gestionar, así como las operaciones que se realizarán sobre ellos. Esta etapa implica la comprensión de las necesidades del negocio o la organización, así como la identificación de los usuarios finales y sus requerimientos.

La segunda etapa es la creación de un modelo conceptual, que es una representación gráfica de las entidades, relaciones y atributos que conformarán la base de datos. Este modelo se basa en la información recopilada durante la etapa de definición de requisitos y sirve como punto de partida para el diseño detallado de la base de datos.

La tercera etapa es el diseño lógico de la base de datos, en la cual se crea una descripción detallada de la estructura de la base de datos en términos de tablas, columnas, claves primarias y foráneas, restricciones de integridad y otros elementos. En esta etapa se consideran aspectos como la normalización de la base de datos para reducir la redundancia y mejorar la integridad de los datos.

La cuarta etapa es el diseño físico de la base de datos, que se enfoca en la implementación del modelo lógico en el sistema de gestión de base de datos seleccionado. Esta etapa implica la definición de los tipos de datos, la creación de índices y la configuración de las opciones de almacenamiento.

Finalmente, la quinta etapa es la implementación de la base de datos, en la cual se crean las tablas, se ingresan los datos iniciales y se realizan pruebas para garantizar que la base de datos funcione correctamente.

Un ejemplo de aplicación del diseño de datos es en el desarrollo de un sistema de gestión de inventarios para una empresa. En la etapa de definición de requisitos, se identifican los datos que se deben recopilar, como el nombre del producto, el número de inventario, la descripción, el precio, la cantidad disponible y la ubicación del almacenamiento. En la etapa de modelado conceptual, se representa gráficamente la relación entre estas entidades y atributos. En el diseño lógico, se definen las tablas, las claves primarias y foráneas, y se normalizan los datos para reducir la redundancia. En el diseño físico, se define el tipo de datos para cada columna y se configuran las opciones de almacenamiento. En la etapa de implementación, se crea la base de datos, se ingresan los datos iniciales y se realizan pruebas para garantizar que la base de datos funcione correctamente.

Diseño lógico estándar

El diseño lógico estándar es una etapa fundamental en el desarrollo de un sistema de información, que tiene como objetivo la creación de un modelo de datos independiente de cualquier tecnología específica de implementación. En otras palabras, es la representación conceptual de los datos, sin considerar cómo se almacenarán físicamente en una base de datos o cómo se manipularán en una aplicación.

Durante esta etapa, se identifican los objetos o entidades relevantes del sistema y sus relaciones, a fin de crear un modelo de datos consistente y preciso. También se definen las restricciones y reglas de negocio que deben aplicarse a los datos. El

resultado de esta etapa es un esquema lógico de la base de datos, que servirá como punto de partida para la implementación física del sistema.

Para diseñar un modelo lógico estándar, se siguen ciertos pasos, que pueden variar dependiendo de la metodología de diseño utilizada. En general, se incluyen las siguientes etapas:

1. Identificación de los requerimientos: se determinan los objetivos del sistema y las necesidades de los usuarios.
2. Análisis de los procesos de negocio: se estudian los procesos de negocio que se quieren automatizar y se identifican los objetos y relaciones entre ellos.
3. Diseño conceptual: se crea un modelo conceptual de los datos, que describe los objetos y sus relaciones.
4. Diseño lógico: se transforma el modelo conceptual en un modelo lógico, que es independiente de cualquier tecnología de base de datos.
5. Normalización: se aplican técnicas de normalización para eliminar redundancias y mejorar la integridad y consistencia de los datos.
6. Revisión y validación: se revisa el diseño lógico para asegurar que cumple con los requerimientos y reglas de negocio, y se validan los datos para asegurar su calidad.

Un ejemplo de diseño lógico estándar puede ser el diseño de una base de datos para un sistema de control de inventarios de una tienda en línea. Se identifican los objetos principales como productos, clientes, proveedores y pedidos, y se establecen las relaciones entre ellos. Se definen las reglas de negocio, como la restricción de cantidad mínima y máxima de productos en un pedido, y se aplica la normalización para evitar redundancias y mejorar la integridad de los datos. El resultado es un modelo lógico estándar que puede ser implementado en cualquier sistema de base de datos.

Diseño lógico específico

El diseño lógico específico es la segunda etapa del proceso de diseño de bases de datos y tiene como objetivo crear un modelo conceptual detallado de la base de datos. En esta etapa, se utilizan las especificaciones del diseño lógico estándar para definir la estructura y las relaciones de la base de datos de manera específica para la aplicación en cuestión.

En el diseño lógico específico, se definen las tablas y sus atributos, las relaciones entre las tablas, las restricciones de integridad, las reglas de negocio y los procedimientos almacenados necesarios para realizar las operaciones de la base de datos. Este proceso implica la transformación de las entidades y relaciones identificadas en el diseño lógico estándar en tablas y relaciones específicas de la aplicación.

Por ejemplo, en una base de datos para una tienda de ropa, se pueden definir tablas para clientes, productos, pedidos y facturas. Cada tabla tendría sus atributos específicos, como nombre, dirección y correo electrónico para la tabla de clientes, y precio, talla y color para la tabla de productos. Además, se definirían las relaciones entre estas tablas, como la relación uno a muchos entre los pedidos y los clientes, y las restricciones de integridad, como la restricción de clave única para el número de factura.

En resumen, el diseño lógico específico es una etapa crucial en el proceso de diseño de bases de datos ya que establece la estructura y las relaciones específicas necesarias para la aplicación en cuestión.

Transformación del Modelo Entidad Relación a Modelo Relacional

La transformación del modelo Entidad-Relación (ER) a Modelo Relacional (MR) es un proceso crucial en el diseño de bases de datos relacionales. El modelo ER es una representación gráfica de entidades y sus relaciones, mientras que el modelo MR es una representación en forma de tablas que describe las entidades, atributos y relaciones entre ellas. El objetivo de la transformación es convertir el modelo ER en una implementación de base de datos relacional.

El proceso de transformación se realiza en varias etapas. Primero, se identifican las entidades y sus atributos en el modelo ER. Luego, se identifican las relaciones entre las entidades y se establecen las claves primarias y foráneas necesarias. A continuación, se normalizan las tablas para reducir la redundancia de datos y garantizar la integridad referencial.

Una vez que se ha realizado la normalización, se pueden crear las tablas en el modelo MR. Cada tabla representa una entidad del modelo ER, y las claves primarias y foráneas se utilizan para establecer las relaciones entre ellas. Los atributos de cada entidad se convierten en columnas de la tabla correspondiente.

Un ejemplo de transformación del modelo ER a MR es el de una base de datos de una tienda en línea. En el modelo ER, las entidades serían "Cliente", "Producto" y "Pedido", con relaciones entre ellas. Por ejemplo, un cliente puede realizar varios pedidos, y cada pedido puede contener varios productos. En la transformación a MR, se crearían tres tablas: "Clientes", "Productos" y "Pedidos". La tabla de clientes tendría columnas para el nombre, dirección, correo electrónico, etc. La tabla de productos tendría columnas para el nombre, descripción, precio, etc. La tabla de pedidos tendría columnas para el número de pedido, la fecha, el cliente que realizó

el pedido y cualquier otra información relacionada con el pedido. Las relaciones entre las tablas se establecerían mediante claves primarias y foráneas

Entidades Fuertes

En el contexto del modelado de datos, una entidad fuerte es un objeto o concepto de la vida real que puede ser identificado de manera única y que puede ser descrito mediante sus atributos o propiedades. En otras palabras, una entidad fuerte es aquella que puede existir por sí misma y que no depende de otras entidades para su existencia.

En el modelo entidad-relación (ER), una entidad fuerte se representa por medio de un rectángulo que contiene el nombre de la entidad, y se identifica por medio de una clave primaria única que permite distinguirla de las demás entidades del modelo. Por ejemplo, en una base de datos de una biblioteca, la entidad fuerte podría ser el libro, que puede ser identificado por su número de ISBN.

Las entidades fuertes son importantes en el diseño de bases de datos ya que proporcionan una forma estructurada de almacenar y recuperar información. Además, permiten la relación entre entidades mediante llaves foráneas, lo que ayuda a establecer las relaciones entre los diferentes elementos de una base de datos.

Otro ejemplo de una entidad fuerte podría ser una empresa en una base de datos de recursos humanos. La empresa tendría atributos como su nombre, dirección, número de identificación fiscal, entre otros, y se podría identificar de manera única mediante su número de identificación fiscal.

En resumen, las entidades fuertes son elementos fundamentales en el modelado de datos, ya que representan objetos o conceptos de la vida real que pueden ser

identificados y descritos mediante sus atributos. Además, permiten establecer las relaciones entre diferentes elementos de una base de datos mediante llaves foráneas.

Entidades Débiles

En el Modelo Entidad-Relación (ER), una entidad débil es una entidad que no puede existir sin la existencia de otra entidad con la cual se relaciona a través de una relación identificadora. Esta relación es conocida como la relación identificadora porque identifica la entidad débil al establecer una relación con la entidad fuerte a la cual está relacionada. Por lo tanto, una entidad débil depende completamente de su relación identificadora para existir, y sin ella, la entidad débil no tiene entidad propia.

En la notación ER, una entidad débil se representa mediante un rectángulo con un doble borde, y una línea punteada conecta la entidad débil con la entidad fuerte a la que está relacionada a través de la relación identificadora. Además, la entidad débil tendrá un atributo que representa su identificador parcial, que es una clave única dentro de la entidad fuerte a la que está relacionada.

Un ejemplo común de una entidad débil es la entidad "Detalle de pedido" en un sistema de gestión de pedidos en línea. La entidad "Pedido" sería la entidad fuerte a la que está relacionada la entidad débil "Detalle de pedido". El detalle de pedido depende de la existencia del pedido al cual está relacionado para tener sentido, ya que no tendría significado alguno sin la existencia del pedido al que pertenece. Además, la entidad "Detalle de pedido" tendría un identificador parcial, que podría ser un número de línea de pedido o un código de producto, que sería único dentro de cada pedido.

En resumen, las entidades débiles son aquellas que no pueden existir sin la existencia de una entidad fuerte y están conectadas a la entidad fuerte a través de una relación identificadora. Estas entidades son comunes en muchos sistemas de base de datos y son importantes para modelar correctamente la realidad que representan.

Atributos Multivalorados

En el diseño de bases de datos, un atributo multivalorado es un tipo de atributo que puede tener varios valores para una misma entidad. Es decir, un atributo multivalorado puede tener más de un valor para una entidad específica. Por ejemplo, un cliente puede tener más de un número de teléfono asociado a su cuenta, o un libro puede tener varios autores.

En un modelo entidad-relación, un atributo multivalorado se representa mediante una elipse con doble línea, mientras que en el modelo relacional se suele crear una tabla separada para almacenar los valores múltiples. Por ejemplo, si un cliente tiene varios números de teléfono, se podría crear una tabla "Teléfonos" con los campos "ID de Cliente" y "Número de teléfono".

Los atributos multivalorados pueden ser de gran utilidad en el diseño de bases de datos, ya que permiten almacenar información adicional y detallada sobre una entidad específica. Sin embargo, también pueden complicar la estructura de la base de datos y dificultar su gestión, por lo que es importante evaluar cuidadosamente si su uso es necesario en cada caso.

En resumen, los atributos multivalorados son aquellos que pueden tener varios valores para una misma entidad. Su uso puede ser útil para almacenar información adicional y detallada, pero también puede complicar la estructura de la base de

datos. Algunos ejemplos comunes de atributos multivalorados son los números de teléfono, las direcciones y los autores de un libro.

Relación 1: 1

En el Modelo Entidad-Relación (MER), una relación 1:1 es aquella en la que una entidad de un tipo se relaciona con otra entidad de otro tipo, de manera que ambas entidades tienen exactamente una instancia relacionada en el otro tipo. Esto significa que una entidad en el lado "uno" de la relación solo puede tener una instancia relacionada en el otro lado de la relación y viceversa.

Un ejemplo de una relación 1:1 podría ser la relación entre un empleado y su número de identificación de seguridad social. Cada empleado solo tiene un número de identificación de seguridad social y cada número de identificación de seguridad social está asociado con un solo empleado. En este caso, la entidad "Empleado" está relacionada con la entidad "Número de Identificación de Seguridad Social" en una relación 1:1.

Otro ejemplo puede ser la relación entre un país y su capital. Cada país tiene una sola capital, y cada capital solo está asociada con un país. Por lo tanto, la entidad "País" está relacionada con la entidad "Capital" en una relación 1:1.

En general, las relaciones 1:1 son menos comunes que las relaciones 1:N y N:M, pero pueden ser útiles en situaciones donde se requiere una relación precisa y exclusiva entre dos entidades.

Relación 1: N

En el modelo entidad-relación, una relación 1:N (uno a muchos) es una asociación entre dos entidades en la que una instancia de una entidad está relacionada con varias instancias de otra entidad. En otras palabras, una entidad puede tener varias instancias relacionadas con una sola instancia de otra entidad.

Por ejemplo, consideremos un modelo de base de datos para una tienda en línea. Una entidad "cliente" está relacionada con la entidad "pedido" en una relación 1:N, ya que un cliente puede realizar varios pedidos, pero un pedido sólo puede pertenecer a un único cliente. En este caso, la entidad "cliente" actúa como la entidad "1" y la entidad "pedido" actúa como la entidad "N".

Otro ejemplo podría ser una relación entre una entidad "departamento" y una entidad "empleado" en una empresa. En este caso, un departamento puede tener varios empleados, pero un empleado sólo puede pertenecer a un departamento. Entonces, la entidad "departamento" actúa como la entidad "1" y la entidad "empleado" actúa como la entidad "N".

En el modelo entidad-relación, una relación 1:N se representa mediante una línea que conecta la entidad "1" con la entidad "N". La línea se etiqueta con "1" en un extremo y "N" en el otro extremo, indicando que la entidad "1" está relacionada con varias instancias de la entidad "N". Además, la relación 1:N también se puede representar en forma de tabla relacional en la que se agrega la clave principal de la entidad "1" como una clave foránea en la tabla de la entidad "N".

Relación M: N

En el diseño de bases de datos, la relación M:N, también conocida como relación muchos-a-muchos, se presenta cuando varios elementos de una tabla pueden estar relacionados con varios elementos de otra tabla. Esta relación se resuelve mediante la creación de una tercera tabla, llamada tabla intermedia o tabla de unión, que relaciona las dos tablas originales.

La tabla de unión incluirá al menos dos columnas, cada una de las cuales se convertirá en una clave externa que se relacionará con la clave primaria de las dos tablas originales. En algunos casos, también puede incluir columnas adicionales para almacenar información adicional sobre la relación entre los elementos.

Un ejemplo común de relación M:N se encuentra en un sistema de gestión de una biblioteca, donde un libro puede ser prestado por varios usuarios y un usuario puede tomar prestados varios libros. En este caso, se crearía una tabla de unión que relacione las tablas de libros y usuarios, que contendría al menos dos columnas: una para el ID del libro y otra para el ID del usuario.

Otro ejemplo puede ser una tienda en línea, donde varios clientes pueden comprar varios productos. En este caso, se crearía una tabla de unión que relacione las tablas de clientes y productos, y puede incluir una columna adicional para almacenar la cantidad de cada producto que se compró.

Es importante diseñar correctamente las relaciones M:N para evitar problemas de redundancia de datos y asegurarse de que se pueda acceder fácilmente a la información relacionada.

Relación n-arios (grado mayor que 2)

En el modelo entidad-relación, una relación n-aria es aquella que involucra a tres o más entidades en una misma relación. A diferencia de las relaciones binarias, que solo involucran a dos entidades, las relaciones n-arias son útiles para modelar situaciones más complejas.

Una relación n-aria se representa mediante un rombo con el nombre de la relación en su interior. Las entidades se conectan a la relación mediante líneas, y cada línea se etiqueta con un rol que indica el papel que juega cada entidad en la relación.

Por ejemplo, en un modelo de una tienda de ropa, se podría tener una relación n-aria entre las entidades "Cliente", "Producto" y "Tienda". La relación podría llamarse "Compra" y cada entidad tendría un rol específico en la relación: el Cliente realiza la Compra de un Producto en una Tienda determinada.

Otro ejemplo podría ser una relación n-aria entre las entidades "Profesor", "Estudiante", "Asignatura" y "Grupo". La relación podría llamarse "Imparte" y cada entidad tendría un rol específico: el Profesor imparte una Asignatura a un Grupo de Estudiantes.

Es importante tener en cuenta que las relaciones n-arias pueden generar cierta complejidad en el modelo y que su uso debe ser justificado por la necesidad de modelar situaciones reales. Además, es recomendable limitar su uso a un grado no mayor de 4 o 5 entidades, para evitar que el modelo se vuelva demasiado complejo y difícil de entender.

Teoría de la Normalización

La teoría de la normalización es una técnica utilizada en la base de datos para reducir la redundancia de datos y garantizar la integridad y consistencia de la información almacenada. Esta teoría establece una serie de reglas o formas normales que deben cumplirse para evitar problemas en la gestión y mantenimiento de los datos.

La normalización se divide en diferentes niveles o formas normales, siendo la primera forma normal (1FN) la más básica y la tercera forma normal (3FN) la más comúnmente utilizada. Cada forma normal establece un conjunto de reglas que deben cumplirse para que una tabla sea considerada normalizada.

Entre las principales ventajas de la normalización se encuentran una mayor eficiencia en la gestión de los datos, una menor redundancia y duplicidad de información, una mayor flexibilidad en la estructuración de la base de datos y una mejor adaptabilidad a cambios futuros en el sistema.

Un ejemplo de aplicación de la teoría de la normalización puede ser en una base de datos de una tienda en línea. Si la tabla de clientes contiene tanto información personal como información de sus compras, se estaría violando la primera forma normal, ya que se estarían mezclando diferentes tipos de información en una misma tabla. Para solucionar este problema, se podría separar la información de clientes y compras en dos tablas diferentes, cada una con su propia clave primaria y referenciando una a la otra mediante una clave foránea. De esta manera, se estaría cumpliendo con la primera forma normal.

Dependencias Funcionales

En el contexto de la teoría de bases de datos, una dependencia funcional es una relación que existe entre dos o más atributos en una tabla, en la que el valor de un atributo determina el valor de otro atributo. En otras palabras, una dependencia funcional indica que el valor de un atributo en una tabla puede ser determinado de manera única por el valor de otro atributo o un conjunto de ellos.

Por ejemplo, en una tabla de empleados, el atributo "nombre del departamento" puede determinarse de manera única por el atributo "número de departamento". En este caso, podemos decir que el atributo "nombre del departamento" depende funcionalmente del atributo "número de departamento".

Las dependencias funcionales son importantes en el diseño de bases de datos porque permiten identificar redundancias en la información y ayudan a mantener la integridad de los datos. La normalización es una técnica que se utiliza para reducir las redundancias en una base de datos y se basa en el análisis de las dependencias funcionales entre los atributos.

En resumen, una dependencia funcional se presenta cuando un atributo o conjunto de atributos en una tabla determina de manera única el valor de otro atributo. El conocimiento de las dependencias funcionales es importante en el diseño de bases de datos para eliminar redundancias y mantener la integridad de los datos.

Primera Forma Normal

La primera forma normal (1FN) es un término utilizado en el diseño de bases de datos relacionales para describir una tabla en la que no hay valores repetidos ni grupos de valores que se repiten. En otras palabras, cada celda de una tabla contiene un solo valor atómico y no hay campos multivalorados.

La 1FN es una condición necesaria para que una tabla pueda ser normalizada en formas superiores. Al cumplir con esta condición, se evitan problemas como la redundancia de datos y la inconsistencia en la actualización de la información.

Por ejemplo, supongamos que se tiene una tabla llamada "Clientes" con los siguientes campos:

- ID de cliente
- Nombre de cliente
- Lista de productos comprados

En este caso, la lista de productos comprados es un campo multivalorado y, por lo tanto, viola la 1FN. En lugar de tener un solo registro para cada cliente, se tendría uno para cada producto que haya comprado. Una solución para cumplir con la 1FN sería crear una tabla adicional para los productos comprados por cada cliente y establecer una relación de uno a muchos entre ambas tablas. De esta manera, se tendría una tabla "Clientes" con los campos "ID de cliente" y "Nombre de cliente", y otra tabla "Productos Comprados" con los campos "ID de cliente" y "Producto comprado".

Segunda Forma Normal

La Segunda Forma Normal (2FN) es una técnica de normalización de datos que se utiliza para garantizar la integridad y la consistencia de los datos en una base de datos relacional. La 2FN se basa en la eliminación de las dependencias parciales en las relaciones que no cumplen con la 1FN.

Una dependencia parcial ocurre cuando un atributo no clave depende solo de una parte de la clave primaria. La eliminación de las dependencias parciales implica dividir una tabla en dos o más tablas para evitar la redundancia de datos y mejorar la calidad de los datos.

Para que una tabla esté en 2FN, debe cumplir dos condiciones: debe estar en 1FN y no debe tener dependencias parciales. En otras palabras, todos los atributos no clave deben depender completamente de la clave primaria. Esto se logra al dividir la tabla en dos o más tablas separadas y crear una relación entre ellas.

Un ejemplo de una tabla que no cumple con la 2FN es una tabla de pedidos que incluye información sobre el cliente, el producto y la fecha del pedido. Si un cliente realiza varios pedidos de diferentes productos, la información del cliente se duplica en cada registro de pedido, lo que puede generar problemas de consistencia y dificultades para mantener la integridad de los datos. En lugar de eso, se puede dividir la tabla en dos tablas separadas, una para los clientes y otra para los pedidos, y crear una relación entre ellas mediante la clave primaria del cliente.

En resumen, la 2FN es una técnica importante para garantizar la calidad de los datos en una base de datos relacional, evitando redundancias y asegurando la integridad de los datos. La eliminación de las dependencias parciales mediante la división de las tablas en relaciones separadas es esencial para cumplir con la 2FN.

Tercera Forma Normal

La tercera forma normal es una forma de normalización utilizada en la teoría de bases de datos relacionales. Esta forma de normalización se enfoca en eliminar las dependencias transitivas en una tabla de base de datos, lo que significa que cualquier columna que no sea una clave primaria debe depender únicamente de la clave primaria y no de ninguna otra columna no clave en la tabla.

En otras palabras, una tabla se encuentra en tercera forma normal si y solo si todas las columnas que no son claves primarias dependen solo de la clave primaria. Esto garantiza que los datos estén almacenados de forma óptima, lo que facilita la recuperación y modificación de los mismos.

Por ejemplo, consideremos una tabla de una base de datos que almacena información de empleados, con los campos 'ID de Empleado', 'Nombre', 'Departamento', 'Descripción del Departamento' y 'Salario'. En este caso, el campo 'Descripción del Departamento' no depende de la clave primaria 'ID de Empleado', sino del campo 'Departamento'. Por lo tanto, la tabla no se encuentra en tercera forma normal.

Para solucionar esto, se podría dividir la tabla en dos, una para la información de empleados y otra para la información del departamento. De esta manera, el campo 'Descripción del Departamento' solo dependería de la clave primaria de la tabla del departamento y no habría dependencias transitivas en ninguna de las tablas, cumpliendo así con la tercera forma normal.

La aplicación de la tercera forma normal en el diseño de bases de datos es fundamental para asegurar la integridad y consistencia de los datos, evitando la redundancia y mejorando la eficiencia en la manipulación de los mismos.

Forma Normal de Boyce-Codd

La Forma Normal de Boyce-Codd (FNBC) es una regla de normalización en el modelado de bases de datos relacionales que garantiza la eliminación de dependencias funcionales anómalas. Es una extensión de la Tercera Forma Normal (3FN), pero más restrictiva, y se considera una de las formas normales más estrictas. La FNBC se basa en la idea de que cada atributo de una relación debe depender únicamente de la clave primaria de la relación. En otras palabras, no debe haber ninguna dependencia funcional no trivial entre los atributos que no sean la clave primaria.

Para entender la FNBC, consideremos un ejemplo de una tabla "Ventas" con atributos como "Producto", "Cliente", "Fecha de venta", "Cantidad vendida" y "Precio". Supongamos que la clave primaria es una combinación de "Producto" y "Fecha de venta". Si el atributo "Cantidad vendida" depende también del atributo "Cliente", entonces se viola la FNBC, ya que "Cantidad vendida" no depende únicamente de la clave primaria. En este caso, deberíamos dividir la tabla en dos relaciones, "Ventas" y "Detalle de ventas", donde la clave primaria de "Detalle de ventas" es una combinación de "Producto", "Fecha de venta" y "Cliente".

La FNBC es importante porque evita problemas de redundancia de datos y puede mejorar la eficiencia de las consultas. Sin embargo, es posible que se requiera una gran cantidad de tablas para cumplir con la FNBC, lo que puede dificultar la comprensión y el mantenimiento de la base de datos. En resumen, la Forma Normal de Boyce-Codd es una regla de normalización que garantiza que los atributos de una relación dependan únicamente de la clave primaria y que no existan dependencias funcionales anómalas. Si se cumplen los requisitos de la FNBC, se logra una estructura de base de datos más eficiente y fácil de mantener.

Cuarta Forma Normal

La Cuarta Forma Normal (4FN) es una forma normal utilizada en la teoría de la normalización de bases de datos relacionales. Fue introducida por Ronald Fagin en 1981 como una forma más restrictiva que la Tercera Forma Normal (3FN), en la que se eliminan las dependencias multi-valuadas y las dependencias no triviales entre los atributos de la clave primaria.

La idea básica de la 4FN es que no debe haber dependencias entre los atributos no clave, salvo aquellas que puedan derivarse de las claves de la relación. Esto significa que una relación que cumple con la 4FN no puede tener dependencias no triviales entre los atributos no clave.

Un ejemplo de aplicación de la 4FN es el siguiente: Supongamos que tenemos una tabla llamada "Estudiantes" con los siguientes campos:

- ID_estudiante
- Nombre
- Dirección
- Cursos_tomados
- Profesores_asociados

Si cada estudiante solo puede tomar cursos con un solo profesor, entonces la tabla cumple con la Tercera Forma Normal, pero no con la Cuarta Forma Normal, ya que los campos "Cursos_tomados" y "Profesores_asociados" están relacionados entre sí pero no son dependientes de la clave primaria "ID_estudiante". Para cumplir con la 4FN, se podría crear dos tablas separadas, una para los cursos tomados por cada estudiante y otra para los profesores asociados con cada estudiante. Cada una de estas tablas tendría una clave primaria que incluiría el ID del estudiante y el ID del curso o del profesor respectivamente.

En resumen, la Cuarta Forma Normal (4FN) busca eliminar las dependencias no triviales entre los atributos de una tabla, permitiendo una mayor independencia y flexibilidad en el manejo de la información. Es importante destacar que no todas las tablas necesitan cumplir con la 4FN, sino que debe evaluarse caso por caso la necesidad de aplicar esta forma normal.

Quinta Forma Normal

La Quinta Forma Normal (5FN) es una forma de normalización de bases de datos relacionales que busca eliminar las dependencias de unión entre relaciones que no son cubiertas por la Cuarta Forma Normal (4FN). La 5FN es una extensión de la 4FN y se considera la forma normal más alta en la teoría de la normalización.

En la 5FN, una relación está en esta forma normal si todas las dependencias de unión implican claves candidatas. Esto significa que cualquier relación que pueda ser dividida en dos o más relaciones que no se pueden unir sin la necesidad de una clave candidata adicional, está en la 5FN.

Un ejemplo de aplicación de la 5FN sería en una base de datos de una empresa que tiene múltiples ubicaciones. Si la empresa tiene una tabla de "Empleados" y una tabla de "Ubicaciones", y cada empleado puede trabajar en varias ubicaciones, se podría crear una tercera tabla de "Empleados en Ubicaciones" que relacione las dos primeras. Sin embargo, si los datos en la tabla "Empleados en Ubicaciones" también contienen información adicional, como la fecha en que el empleado comenzó a trabajar en una determinada ubicación, entonces la tabla puede violar la 5FN. En este caso, se podría separar la tabla en dos tablas diferentes: una tabla que relacione empleados y ubicaciones, y otra tabla que registre la información adicional sobre la fecha de inicio de trabajo de cada empleado en cada ubicación.

En resumen, la Quinta Forma Normal es una forma de normalización que busca eliminar todas las dependencias de unión que no estén cubiertas por la Cuarta Forma Normal. Su objetivo es minimizar la redundancia de datos y aumentar la integridad de la base de datos.

Bibliografía y Webgrafía

Olarte, C. A., Ocampo, E., & otros. (2017). Diseño y gestión de bases de datos. Pearson Educación.

Joyanes Aguilar, L. (2013). Bases de Datos: Desde Chen hasta Codd con Oracle. McGraw Hill.

Piattini Velthuis, M. G., Calero, C., Genero, M., & Ruiz, F. (2015). Diseño y administración de bases de datos. Alfaomega.

Alonso, R., & Vizcaíno, A. (2013). Bases de datos. Paraninfo.

Glosario

Atributos Multivalorados: son atributos que pueden tener varios valores al mismo tiempo en una misma entidad.

Cuarta Forma Normal: es una forma normal utilizada en bases de datos para minimizar la redundancia de datos, y asegurar la consistencia y estabilidad de la base de datos.

Dependencias Funcionales: son relaciones entre atributos en una base de datos, en la que un conjunto de atributos determina de manera única otro conjunto de atributos.

Diseño lógico específico: es una etapa del diseño de bases de datos en la que se establecen los detalles específicos de cómo se implementarán las tablas y relaciones en la base de datos.

Diseño lógico estándar: es una etapa del diseño de bases de datos en la que se establece una estructura general de la base de datos, incluyendo las tablas, atributos y relaciones entre ellas.

Entidades Débiles: son entidades que dependen de otra entidad para existir, y se identifican por su relación con una entidad fuerte.

Entidades Fuertes: son entidades que pueden existir por sí mismas en una base de datos, sin necesidad de depender de otras entidades.

Forma Normal de Boyce-Codd: es una forma normal utilizada en bases de datos para asegurar que no existan dependencias funcionales triviales, lo que puede provocar redundancia de datos.

Primera Forma Normal: es una forma normal utilizada en bases de datos para asegurar que cada atributo tenga un solo valor en una entidad.

Relación 1:1: es una relación entre dos entidades en la que una entidad tiene una relación única con otra entidad.

Relación 1:N: es una relación entre dos entidades en la que una entidad puede tener varias relaciones con otra entidad, pero esta última solo tiene una relación con la primera entidad.

Relación M:N: es una relación entre dos entidades en la que una entidad puede tener varias relaciones con varias entidades de la otra entidad.

Relación n-aria: es una relación entre tres o más entidades en una base de datos.

Segunda Forma Normal: es una forma normal utilizada en bases de datos para asegurar que cada atributo de una entidad esté relacionado directamente con la clave principal de la entidad.

Teoría de la Normalización: es el proceso de estructurar una base de datos de manera que se minimice la redundancia de datos y se asegure la integridad de los datos.

Tercera Forma Normal: es una forma normal utilizada en bases de datos para asegurar que no existan dependencias transitivas, lo que puede provocar redundancia de datos.

Transformación del Modelo Entidad Relación a Modelo Relacional: es el proceso de convertir un modelo de entidad relación a una base de datos relacional.